

Layout Managers

Layout Manager	Arrangement
FlowLayout	Left to right
BorderLayout	North, South, East, West, Center
GridLayout	Rows and columns
CardLayout	One component/card at a time
GridBagLayout	Flexible grid arrangement

FlowLayout

Components are arranged **from left to right**. When there is no space, they move to the next line.

```
import java.awt.*;

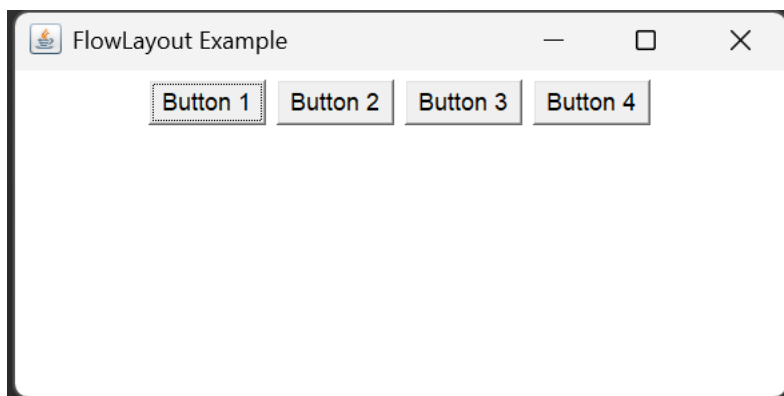
public class FlowExample {
    public static void main(String[] args) {

        Frame f = new Frame("FlowLayout Example");

        f.setLayout(new FlowLayout());

        f.add(new Button("Button 1"));
        f.add(new Button("Button 2"));
        f.add(new Button("Button 3"));
        f.add(new Button("Button 4"));

        f.setSize(400, 200);
        f.setVisible(true);
    }
}
```



BorderLayout

The window is divided into **5 regions**:

- North
- South
- East
- West
- Center

```
import java.awt.*;

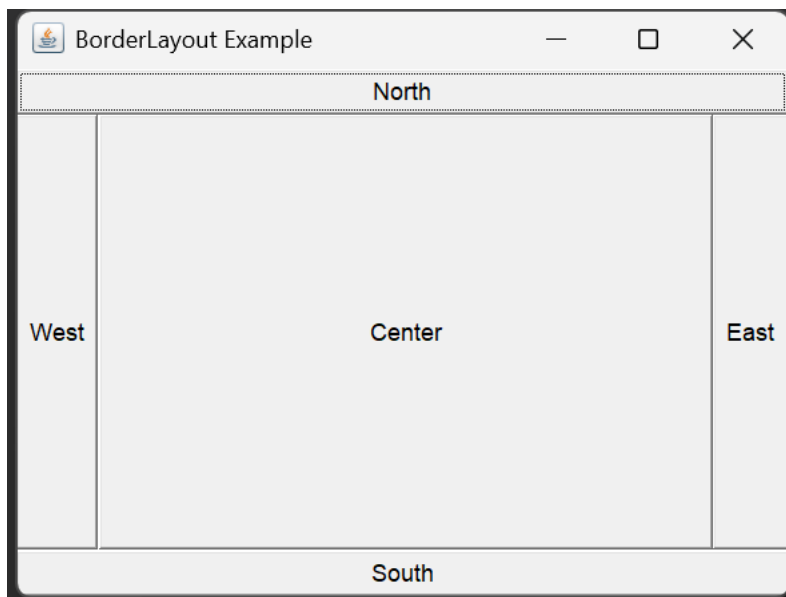
public class BorderExample {
    public static void main(String[] args) {

        Frame f = new Frame("BorderLayout Example");

        f.setLayout(new BorderLayout());

        f.add(new Button("North"), BorderLayout.NORTH);
        f.add(new Button("South"), BorderLayout.SOUTH);
        f.add(new Button("East"), BorderLayout.EAST);
        f.add(new Button("West"), BorderLayout.WEST);
        f.add(new Button("Center"), BorderLayout.CENTER);

        f.setSize(400, 300);
        f.setVisible(true);
    }
}
```



GridLayout

Components are arranged in **rows and columns**, like a table.

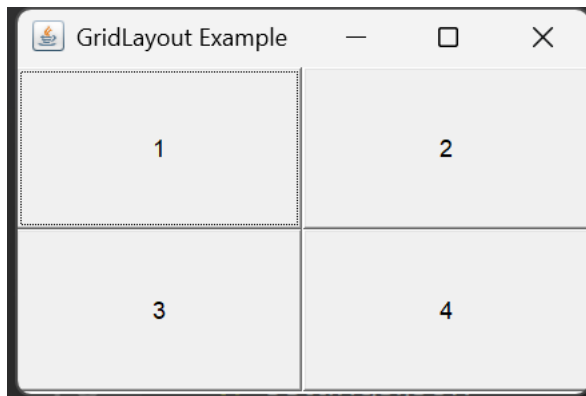
```
import java.awt.*;
public class GridExample {
    public static void main(String[] args) {

        Frame f = new Frame("GridLayout Example");

        f.setLayout(new GridLayout(2, 2));

        f.add(new Button("1"));
        f.add(new Button("2"));
        f.add(new Button("3"));
        f.add(new Button("4"));

        f.setSize(300, 200);
        f.setVisible(true);
    }
}
```



CardLayout

Only one component/card is visible at a time. We can move from one card to another.

```
import java.awt.*;
import java.awt.event.*;

public class CardExample extends Frame implements ActionListener {

    CardLayout card;
    Panel cards;
    Button next;

    CardExample() {
        card = new CardLayout();
        // Panel containing the cards
    }
}
```

```
cards = new Panel();
cards.setLayout(card);

// Card 1
Panel p1 = new Panel();
p1.setBackground(Color.RED);
p1.add(new Label("CARD 1"));
cards.add(p1, "one");

// Card 2
Panel p2 = new Panel();
p2.setBackground(Color.GREEN);
p2.add(new Label("CARD 2"));
cards.add(p2, "two");

// Card 3
Panel p3 = new Panel();
p3.setBackground(Color.BLUE);
p3.add(new Label("CARD 3"));
cards.add(p3, "three");

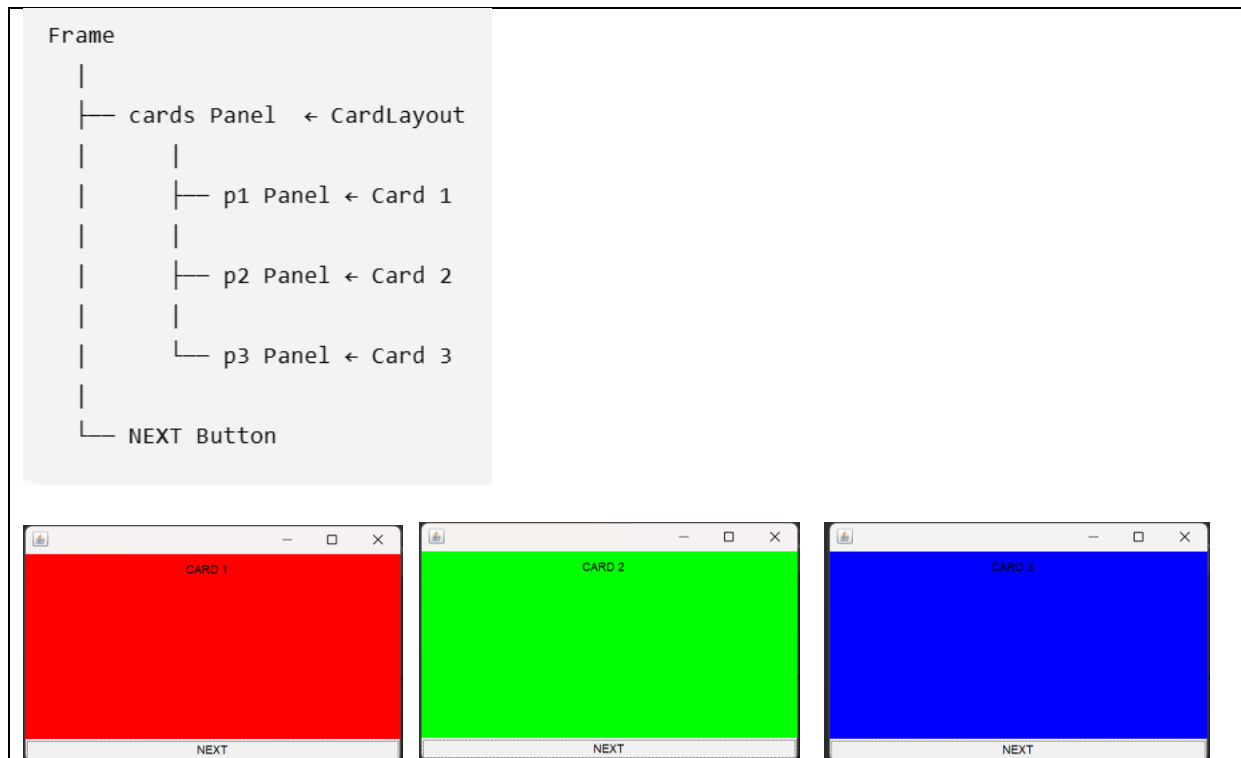
// Next button
next = new Button("NEXT");
next.addActionListener(this);

add(cards, BorderLayout.CENTER);
add(next, BorderLayout.SOUTH);

setSize(400, 250);
setVisible(true);
}

public void actionPerformed(ActionEvent e) {
    card.next(cards);
}

public static void main(String[] args) {
    new CardExample();
}
}
```



GridBagLayout

It allows components to be placed in a **flexible grid**. It is more powerful but more complicated.

```
import java.awt.*;

public class GridBagExample {
    public static void main(String[] args) {

        Frame f = new Frame("GridBagLayout Example");

        f.setLayout(new GridBagLayout());

        GridBagConstraints gbc = new GridBagConstraints();
        // controls the position and size of each component.

        // Button 1 - left side, occupies 2 rows
        gbc.gridx = 0;
        gbc.gridy = 0;
        gbc.gridheight = 2;
        gbc.fill = GridBagConstraints.BOTH;
        f.add(new Button("1"), gbc);

        // Button 2 - top right
        gbc.gridx = 1;
        gbc.gridy = 0;
        gbc.gridheight = 1;
```

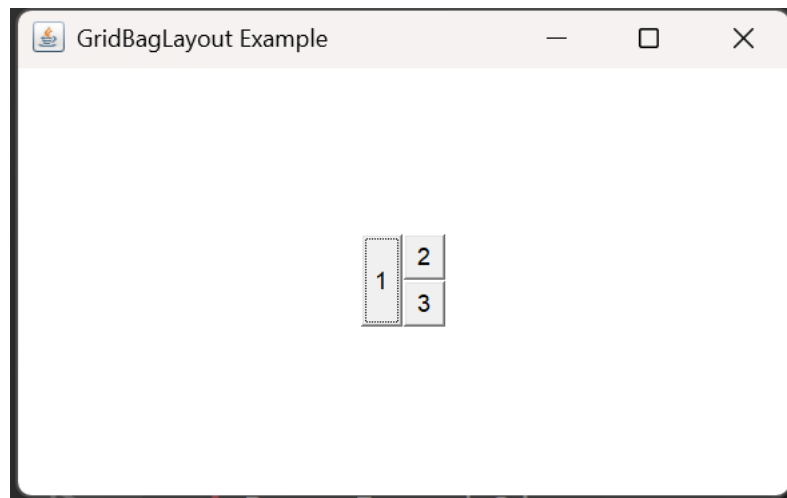
```

f.add(new Button("2"), gbc);

// Button 3 - bottom right
gbc.gridx = 1;
gbc.gridy = 1;
f.add(new Button("3"), gbc);

f.setSize(400, 250);
f.setVisible(true);
}
}

```



GridLayout	GridBagLayout
Divides the container into equal-sized cells .	Divides the container into a flexible grid .
All components get equal width and height .	Components can have different sizes .
Simple to use.	More complex to use.
Uses only rows and columns .	Uses GridBagConstraints to control position and size.
Components occupy one cell each .	A component can occupy multiple rows or columns .
Less control over component placement.	Provides more control over component placement.
Suitable for calculator/keypad-like layouts .	Suitable for forms and complex GUI designs .